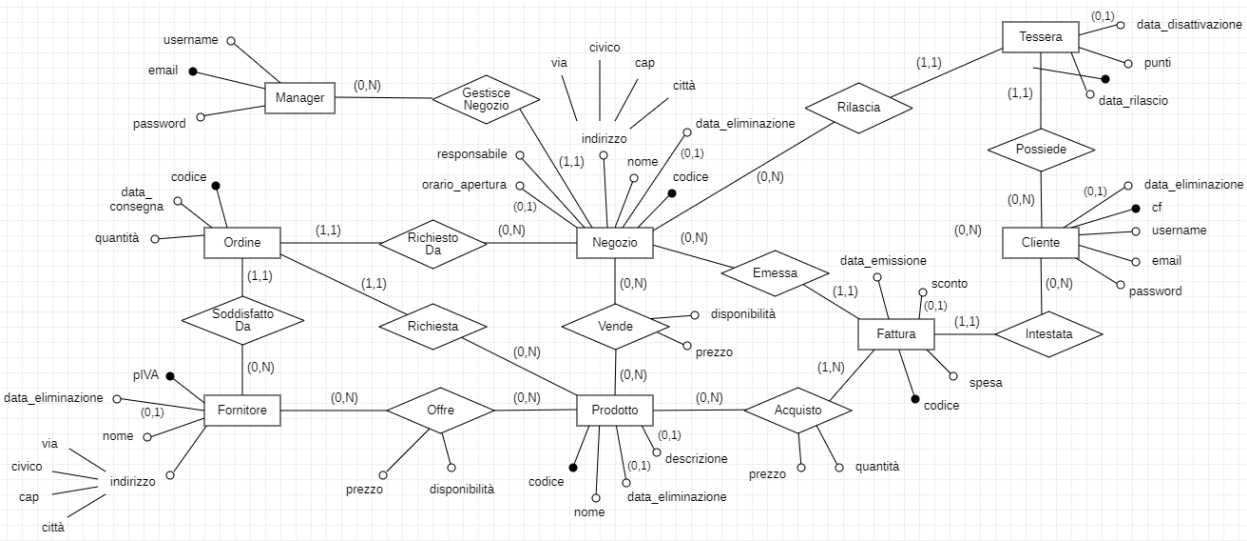
Documentazione Tecnica

Schema E-R



Vincoli Extra-Schema:

- Un cliente può avere più di una tessera associata, ma solo una tra queste deve essere attiva.
- Ciascuna fattura deve essere associata ad almeno una record di Acquisto

Commento dello Schema E-R

- Gestisce Negozio (Manager — Negozio)
  un manager può gestire più negozi → (0,N)
  un negozio deve essere gestito solamente da un manager → (1,1)
- Vende (Negozio — Prodotto) , con attributi disponibilità e prezzo
  un negozio mette in vendita più prodotti → (0,N)
  un prodotto può essere venduto in più negozi → (0,N)
- Offre (Fornitore — Prodotto) , con attributi disponibilità e prezzo
  un fornitore offre ai negozi più prodotti → (0,N)
  un prodotto può essere offerto ai negozi da più fornitori → (0,N)
- Richiesto Da (Ordine — Negozio)
  un ordine deve essere richiesto solamente da un negozio → (1,1)
  un negozio può effettuare più ordini → (0,N)
- Soddisfatto Da (Ordine — Fornitore)
  un ordine deve essere soddisfatto solamente da un fornitore → (1,1)
  un fornitore può soddisfare più ordini → (0,N)

- Richiesta (Ordine — Prodotto)
  un ordine deve contenere solamente un prodotto → (1,1)
  un prodotto può essere richiesto in più ordini → (0,N)
- Rilascia (Tessera — Negozio)
  una tessera viene rilasciata da un solo negozio → (1,1)
  un negozio può rilasciare più tessere → (0,N)
- Possiede (Tessera - Cliente)
  una tessera deve essere posseduta solamente da un cliente → (1,1)
  un cliente può possedere più di una tessera, vedere vincoli extra-schema → (0,N)
- Emessa (Fattura - Negozio)
  una fattura deve essere emessa solamente da un negozio → (1,1)
  un negozio può emettere più fattura → (0,N)
- Intestata (Fattura - Cliente)
  una fattura deve essere intestata solamente ad un cliente → (1,1)
  ad un cliente possono essere intestate più fatture → (0,N)
- Acquisto (Fattura - Prodotto), con attributi quantità e prezzo
  una fattura contiene almeno un prodotto acquistato, vedere vincoli extra-schema → (1,N)
  un prodotto può essere contenuto in più fatture → (0,N)

Essendo le relazioni Vende, Offre e Acquisto delle relazioni di tipo molti a molti, queste verranno trasformate in tabelle nello schema logico.

Schema Logico

Per migliorare la comprensione della base di dati rappresento lo schema logico come il codice DDL utilizzato per la creazione.
Notare che le password sono memorizzate in chiaro per semplicità, ma in un'eventuale implementazione reale dovrebbero essere cifrate.

```
CREATE TABLE manager (
    codice CHAR(8) PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT null,
  email VARCHAR(100) UNIQUE NOT null,
  password VARCHAR(30) NOT null,
  CHECK (email ~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
);

CREATE TABLE negozio (
    codice CHAR(8) PRIMARY KEY,
    nome VARCHAR(50) NOT NULL,
    responsabile VARCHAR(50) NOT NULL,
    orario_apertura TIME,
    via VARCHAR(100) NOT NULL,
    civico VARCHAR(4) NOT NULL,
```

```
    cap CHAR(5) NOT NULL,
  citta VARCHAR(50) NOT NULL,
  data_eliminazione TIMESTAMP,
  manager CHAR(8) NOT null,
  foreign key (manager) references manager(codice)
);

CREATE TABLE cliente (
    codice CHAR(8) PRIMARY KEY,
  cf CHAR(16) UNIQUE NOT NULL,
  username VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password VARCHAR(30) NOT NULL,
  data_eliminazione TIMESTAMP,
  CHECK (cf ~ '^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]$'),
  CHECK (email ~ '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
);

CREATE TABLE tessera (
    codice CHAR(8) PRIMARY KEY,
  data_rilascio TIMESTAMP NOT NULL DEFAULT now(),
  punti INT NOT NULL DEFAULT 0,
  data_disattivazione TIMESTAMP,
  cliente CHAR(8) NOT NULL,
  negozio CHAR(8) NOT NULL,
  FOREIGN KEY (cliente) REFERENCES cliente(codice),
  FOREIGN KEY (negozio) REFERENCES negozio(codice),
  CHECK (punti >= 0)
);

CREATE TABLE fornitore (
    codice CHAR(8) PRIMARY KEY,
    nome VARCHAR(50) NOT NULL,
  pIVA CHAR(11) UNIQUE NOT NULL,
  via VARCHAR(100) NOT NULL,
  civico VARCHAR(4) NOT NULL,
  cap CHAR(5) NOT NULL,
  citta VARCHAR(50) NOT NULL,
  data_eliminazione TIMESTAMP,
  CHECK (pIVA ~ '^[0,9]{11}$')
);

CREATE TABLE prodotto (
  codice CHAR(8) PRIMARY KEY,
  nome VARCHAR(100) NOT NULL UNIQUE,
```

```
  data_eliminazione TIMESTAMP,
  descrizione TEXT
);

CREATE TABLE fattura (
  codice CHAR(8) PRIMARY KEY,
  data_emissione TIMESTAMP NOT NULL DEFAULT now(),
  sconto VARCHAR(2),
  spesa DECIMAL(7,2) NOT NULL DEFAULT 0,
  negozio CHAR(8) NOT NULL,
  cliente CHAR(8) NOT NULL,
  FOREIGN KEY (negozio) REFERENCES negozio(codice),
  FOREIGN KEY (cliente) REFERENCES cliente(codice),
  CHECK (sconto in ('5', '15', '30')),
  CHECK (spesa >= 0)
);

create table ordine (
    codice CHAR(8) PRIMARY KEY,
    data_consegna TIMESTAMP NOT NULL DEFAULT now(),
    quantita INT NOT NULL DEFAULT 1,
    negozio CHAR(8) NOT NULL,
    fornitore CHAR(8) NOT NULL,
    prodotto CHAR(8) NOT NULL,
    FOREIGN KEY (negozio) REFERENCES negozio(codice),
    FOREIGN KEY (fornitore) REFERENCES fornitore(codice),
    FOREIGN KEY (prodotto) REFERENCES prodotto(codice),
    CHECK (quantita > 0)
)

create table vende (
    negozio CHAR(8) NOT NULL,
    prodotto CHAR(8) NOT NULL,
    prezzo DECIMAL(7,2) NOT NULL,
    disponibilita INT DEFAULT 1,
    PRIMARY KEY (negozio, prodotto),
    FOREIGN KEY (negozio) REFERENCES negozio(codice),
  FOREIGN KEY (prodotto) REFERENCES prodotto(codice),
    CHECK (prezzo >= 0),
    CHECK (disponibilita > 0)
)

create table offre (
    fornitore CHAR(8) NOT NULL,
    prodotto CHAR(8) NOT NULL,
```

```
    prezzo DECIMAL(7,2) NOT NULL DEFAULT 0,
    disponibilita INT DEFAULT 1,
    PRIMARY KEY (fornitore, prodotto),
    FOREIGN KEY (fornitore) REFERENCES fornitore(codice),
    FOREIGN KEY (prodotto) REFERENCES prodotto(codice),
    CHECK (prezzo >= 0)
    CHECK (disponibilita > 0)
)

create table acquisto (
    fattura CHAR(8) NOT NULL,
    prodotto CHAR(8) NOT NULL,
    prezzo DECIMAL(7,2) NOT NULL,
    quantita INT NOT NULL DEFAULT 1,
    PRIMARY KEY (fattura, prodotto),
    FOREIGN KEY (fattura) REFERENCES fattura(codice),
    FOREIGN KEY (prodotto) REFERENCES prodotto(codice),
    CHECK (quantita > 0),
    CHECK (prezzo >= 0)
)
```

Funzioni e Trigger

Generazione dei Codici

Per semplificare la comprensione delle entità ho adottato dei codici personalizzati per ciascun entità principale. Tramite l'uso di sequenze riusciamo a tenere una variabile di count globale che ad ogni inserimento incrementa il valore.

```
CREATE SEQUENCE cliente_seq START 1;
```

Prendendo come esempio l'entità `cliente` un codice di esempio é: "CL000001"

La generazione del codice personalizzato avviene ad ogni inserimento tramite un trigger, ogni entità ha il proprio.

```
CREATE OR REPLACE FUNCTION myshop.set_codice_cliente()
 RETURNS trigger AS $$
begin
    new.codice := genera_codice('cliente_seq', 'CL');
    return new;
end;
$$ LANGUAGE plpgsql;
```

```
create trigger i_trg_codice_cliente before insert on
   myshop.cliente for each row execute function myshop.set_codice_cliente();
```

La funzione `genera_codice` é una funzione utilitaria che dato il nome della sequenza e le iniziali dell'entità restituisce il nuovo codice:

```
CREATE OR REPLACE FUNCTION myshop.genera_codice(sequenza text, prefisso text)
 RETURNS text AS $$
declare
   next_id bigint;
begin
   next_id := nextval(sequenza);
   return prefisso || LPAD(next_id::text, 6, '0');
end;
$$ LANGUAGE plpgsql;
```

Ordine Prodotto da Fornitore

Per rifornire il negozio questo può inserire un ordine presso un fornitore. L'ordine é stato implementato su un singolo prodotto per volta, specificandone la quantità che vogliamo acquistare.

Quando un negozio vuole acquistare un prodotto in una determinata quantità, l'ordine viene inserito al fornitore che offre il prodotto a prezzo minore ed ha una disponibilità maggiore.
In questo caso é quindi adatto utilizzare una `select` con `limit 1`, poiché non siamo interessati al caso in cui ci siano più fornitori con stesso prezzo e disponibilità, ci va bene uno casuale tra i due.

```
CREATE OR REPLACE FUNCTION myshop.ordina_prodotto(
   given_negozio character, given_prodotto character, quantita integer)
 RETURNS void AS $$
declare
   id_fornitore myshop.fornitore.codice%TYPE;
begin
   raise notice 'Richiesta di ordine da % con quantita % di % in svolgimento',
      given_negozio, quantita, given_prodotto;

   select fornitore into id_fornitore
      from offre o inner join fornitore f on o.fornitore = f.codice
     where o.prodotto = given_prodotto and o.disponibilita >= quantita
         and f.data_eliminazione is null
     order by o.prezzo asc, o.disponibilita desc limit 1;

   if found then
      raise notice 'Trovato fornitore %', id_fornitore;
```

```
        insert into ordine (quantita, negozio, fornitore, prodotto)
            values (quantita, given_negozio, id_fornitore, given_prodotto);
    else
        raise exception 'Nessun fornitore ha almeno % di %',
            quantita, given_prodotto;
    end if;

end;
$$ LANGUAGE plpgsql;
```

All'inserimento di un ordine la disponibilità del prodotto nel negozio e nel fornitore deve essere aggiornata. Questo viene gestito tramite il seguente trigger:

```
CREATE OR REPLACE FUNCTION myshop.aggiorna_disp_ordine() RETURNS trigger AS $$
declare
    disp_fornitore myshop.offre.disponibilita%TYPE;
    prezzo_fornitore myshop.offre.prezzo%TYPE;
begin
    raise notice 'Nuovo ordine: negozio %, fornitore %, prodotto %',
        new.negozio, new.fornitore, new.prodotto;

    select disponibilita, prezzo into disp_fornitore, prezzo_fornitore
        from offre where fornitore = new.fornitore and prodotto = new.prodotto;

    if disp_fornitore - new.quantita = 0 then
        delete from offre where fornitore = new.fornitore and prodotto = new.prodotto;
    elsif disp_fornitore - new.quantita < 0 then
        raise exception 'Sono stati acquistati % prodotti, ma la disponibilità era di %',
            new.quantita, disp_fornitore;
    else
        update offre set disponibilita = disp_fornitore - new.quantita
            where fornitore = new.fornitore and prodotto = new.prodotto;
    end if;

    perform 1 from vende where negozio = new.negozio and prodotto = new.prodotto;

    if found then
        update vende set disponibilita = disponibilita + new.quantita
            where negozio = new.negozio and prodotto = new.prodotto;
    else
        insert into vende (negozio, prodotto, prezzo, disponibilita)
        values (new.negozio, new.prodotto, prezzo_fornitore, new.quantita);
    end if;

  return new;
```

```
end;
$$ LANGUAGE plpgsql;

create trigger i_ordine_prodotto after insert on
   myshop.ordine for each row execute function myshop.aggiorna_disp_ordine()
```

Acquisto Prodotto da Negozio

Un cliente può acquistare dei prodotti venduti da un negozio. L'acquisto prevede la presenze di 1 o più prodotti, tutti in vendita dallo stesso negozio.

Quando un cliente acquista dei prodotti, per ciascuno di essi viene specificata la quantità, memorizzata assieme al prezzo di vendita in quel momento. Per passare i dati tra back-end e base di dati ho utilizzato il formato json.

L'acquisto é stato implementato tramite una funzione, in modo da rendere il tutto atomico, così da poter tornare allo stato precedente della base di dati in caso di errori

Questa funzione inserisce inizialmente la fattura senza specificare l'attributo `spesa`, che viene quindi impostato di default a 0. Per ogni prodotto acquistato si registrano i dati nella tabella `aquisto` e la variabile `spesa_totale` accumula i prezzi di questi.
Alla fine della funzione si controlla che almeno un prodotto é stato inserito tramite la variabile `num_acquisti` (in questo modo sono in grado di soddisfare il vincolo extra-schema) e si effettua un update su `fattura` per aggiornare il valore dell'attributo `spesa`.

```
CREATE OR REPLACE FUNCTION myshop.nuovo_acquisto(
   given_sconto character varying, given_negozio character,
   given_cliente character, given_acquisti json)
 RETURNS void AS $$
declare
   r_acquisto record;
   codice_fattura myshop.fattura.codice%type;
   spesa_totale myshop.fattura.spesa%type := 0;
   num_acquisti integer := 0;
   disp_prodotto myshop.vende.disponibilita%type;
begin
   raise notice 'Nuovo acquisto nel negozio % dal cliente %',
      given_negozio, given_cliente;

   insert into myshop.fattura (sconto, negozio, cliente)
   values (given_sconto, given_negozio, given_cliente)
   returning codice into codice_fattura;

   FOR r_acquisto IN SELECT * FROM json_to_recordset(given_acquisti)
      AS r(prodotto bpchar(8), quantita integer, prezzo decimal(7,2)) LOOP

      perform 1 from myshop.negozio where codice = given_negozio
```

```
            and data_eliminazione is null;

        if not found then
            raise exception 'Il negozio % esiste', given_negozio;
        end if;

        select v.disponibilita into disp_prodotto
        from myshop.vende v inner join myshop.prodotto p on v.prodotto = p.codice
            and p.data_eliminazione is null
        where v.negozio = given_negozio and v.prodotto = r_acquisto.prodotto
            and v.disponibilita >= r_acquisto.quantita;

        if not found then
            raise exception 'Il negozio % non ha sufficiente disponibilità per
                soddisfare la richiesta del prodotto %', given_negozio, r_acquisto.prodotto;
        end if;

        insert into myshop.acquisto (fattura, prodotto, prezzo, quantita)
        values (codice_fattura, r_acquisto.prodotto, r_acquisto.prezzo,
            r_acquisto.quantita);

            r_acquisto.prodotto, r_acquisto.prezzo, r_acquisto.quantita;

        if disp_prodotto - r_acquisto.quantita = 0 then
            delete from vende where negozio = given_negozio
                and prodotto = r_acquisto.prodotto;
        else
            update myshop.vende set disponibilita = disponibilita - r_acquisto.quantita
                where negozio = given_negozio and prodotto = r_acquisto.prodotto;
        end if;
        spesa_totale := spesa_totale + r_acquisto.prezzo * r_acquisto.quantita;
        num_acquisti := num_acquisti + 1;
    end loop;

    if num_acquisti = 0 then
        raise exception 'La fattura deve essere associata ad almeno un acquisto';
    end if;

    spesa_totale := myshop.applica_sconto(given_sconto, spesa_totale);
    update myshop.fattura set spesa = spesa_totale where codice = codice_fattura;
end;
$$ LANGUAGE plpgsql;
```

Lo sconto sulla spesa totale della fattura viene applicato dalla funzione:

`applica_sconto(sconto character varying, totale numeric)`

Questa calcola lo sconto effettivo che sarà tolto dalla somma dei prezzi di tutti i prodotti, con uno sconto effettivo massimo di 100€.

All'inserimento di una fattura bisogna decurtare i punti utilizzati per lo sconto ed aggiungere i punti guadagnati alla tessera del cliente. Questo é gestito dal seguente trigger:

```
CREATE OR REPLACE FUNCTION myshop.aggiorna_punti_tessera() RETURNS trigger AS $$
begin
   raise notice 'Nuova fattura: utente %, spesa %', new.cliente, new.spesa;

   perform 1 from tessera where cliente = new.cliente and data_disattivazione is null;
   if found then
      if new.sconto is not null then
         raise notice 'Il cliente ha usato uno sconto del %%%', new.sconto;
         update tessera set punti = punti - myshop.punti_sconto(new.sconto)
            where cliente = new.cliente and data_disattivazione is null;
      end if;

      update tessera set punti = punti + floor(new.spesa)
         where cliente = new.cliente and data_disattivazione is null;
      raise notice 'Aggiunto % punti sulla tessera', floor(new.spesa);
   else
      raise warning 'Il cliente % non ha una tessera', new.cliente;
   end if;
   return new;
end;
$$LANGUAGE plpgsql;

create trigger u_fattura_punti after update on
   myshop.fattura for each row execute function myshop.aggiorna_punti_tessera()
```

La funzione `punti_sconto` traduce la stringa contenente lo sconto percentuale in punti spesi.

Soft-Delete

L'eliminazione delle entità principali della base di dati é implementata tramite la tecnica soft-delete, ovvero si tramite l'attributo `data_eliminazione`, se `NOT NULL` allora l'entità é stata eliminata.
Le entità in questione sono cliente, negozio, fornitore e prodotto.

Si può eliminare una tra queste utilizzando la seguente funzione, facile da implementare grazie alla presenza di codici personalizzati:

```
CREATE OR REPLACE FUNCTION myshop.soft_delete(e_codice character) RETURNS boolean AS $$
declare
   prefisso VARCHAR(2);
   tabella TEXT;
```

```
begin
   raise info 'Eliminazione di %', e_codice;

   prefisso := substring(e_codice, 1, 2);

   case prefisso
      when 'NE' then tabella := 'negozio';
      when 'CL' then tabella := 'cliente';
      when 'FO' then tabella := 'fornitore';
      when 'PR' then tabella := 'prodotto';
      else
         raise warning 'Tentativo di eliminazione di un entità non valida: %', e_codice;
         return false;
   end case;

   execute format('update %I set data_eliminazione = current_timestamp
      where codice = %L and data_eliminazione is null', tabella, e_codice);
   return true;
end;
$$ LANGUAGE plpgsql;
```

All'eliminazione di cliente o negozio corrisponde la disattivazione delle tessere a loro associate. Sono infatti definiti i seguenti due trigger:

```
CREATE OR REPLACE FUNCTION myshop.tessere_delete_cliente() RETURNS trigger AS $$
BEGIN
   IF OLD.data_eliminazione IS NULL AND NEW.data_eliminazione IS NOT NULL THEN
      UPDATE tessera SET data_disattivazione = now()
         WHERE cliente = NEW.codice;

      RAISE NOTICE 'Tessera associata al cliente % disattivata', NEW.codice;
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

create trigger u_trg_tessere_delete_cliente after update on
   myshop.cliente for each row execute function myshop.tessere_delete_cliente()
```

```
CREATE OR REPLACE FUNCTION myshop.tessere_delete_negozio() RETURNS trigger AS $$
BEGIN
   IF OLD.data_eliminazione IS NULL AND NEW.data_eliminazione IS NOT NULL THEN
      UPDATE Tessera SET data_disattivazione = now()
            WHERE negozio = NEW.codice;
```

```
        RAISE NOTICE 'Tessere associate al negozio % disattivate', NEW.codice;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

create trigger u_trg_tessere_delete_negozio after update on
    myshop.negozio for each row execute function myshop.tessere_delete_negozio()
```

Controllo Inserimento Tessera

Per soddisfare il vincolo extra-schema sull'associazione di più tessere ad un cliente, all'inserimento di una nuova tessera devo sempre controllare che il cliente in causa non abbia già una tessera attiva.
Il controllo viene effettuato dal seguente trigger:

```
CREATE OR REPLACE FUNCTION myshop.check_insert_tessera() RETURNS trigger AS $$
begin
    raise notice 'Inserimento di una nuova tessera del cliente % al negozio %',
        new.cliente, new.negozio;

    perform 1 from myshop.tessera t where t.cliente = new.cliente
        and t.data_disattivazione is null;

    if found then
        raise exception 'Il cliente % ha già una tessera attiva', new.cliente;
    else
        return new;
    end if;
end;
$$ LANGUAGE plpgsql;

create trigger i_trg_check_tessera before inserton
    myshop.tessera for each row execute function myshop.check_insert_tessera()
```

Storici e Liste

La base di dati offre 3 funzioni di interrogazione per accedere alle informazioni sui prodotti che vendono negozi e fornitori, oltre alle tessere rilasciate dai negozi:

- lista_prodotti_in_vendita(given_negozio character) RETURNS SETOF myshop.prodotti_in_vendita
- lista_prodotti_offerti(given_fornitore character) RETURNS SETOF myshop.prodotti_in_vendita
- lista_tesserati(given_negozio character) RETURNS SETOF myshop.clienti_tesserati

Prendiamo come esempio la prima, le restanti due sono simili, cambiano i dati restituiti ma il corpo é lo stesso:

```
CREATE TYPE myshop.prodotti_in_vendita AS (
    codice bpchar(8),
    nome varchar(100),
    prezzo numeric(7,2),
    disponibilita int4
);

CREATE OR REPLACE FUNCTION myshop.lista_prodotti_in_vendita(given_negozio character)
 RETURNS SETOF myshop.prodotti_in_vendita AS $$
begin
    raise notice 'Richiesta dei prodotti in vendita del negozio %', given_negozio ;
    return query
    SELECT p.codice, p.nome, v.prezzo, v.disponibilita
    FROM prodotto p inner join vende v on p.codice = v.prodotto
        and v.negozio = given_negozio
    WHERE p.data_eliminazione is null
    order by disponibilita asc, p.nome asc;
end;
$$ LANGUAGE plpgsql;
```

Oltre alle precedenti funzioni la base di dati offre anche funzioni di interrogazione per accedere alle informazioni di storici degli acquisti tra fornitori, negozi e clienti.

- myshop.storico_acquisti(given_cliente character) RETURNS SETOF myshop.acquisti_cliente
- myshop.storico_fatture(given_negozio character) RETURNS SETOF myshop.acquisti_cliente
- myshop.storico_ordini(given_fornitore character) RETURNS SETOF myshop.ordine_fornitore

Anche in questo caso prendiamo come esempio la prima, le restanti due sono simili:

```
CREATE OR REPLACE FUNCTION myshop.storico_acquisti(given_cliente character)
 RETURNS SETOF myshop.acquisti_cliente AS $$
begin
    raise notice 'Richiesta dello storico degli acquisti del cliente %', given_cliente;

    return query
    select spesa, c.username, n.nome, data_emissione, (
        select json_agg(
            json_build_object(
                'nome', p.nome,
                'prezzo', a.prezzo,
                'quantita', a.quantita
            )
```

```
        )
    ) as prodotti
    from fattura f inner join acquisto a on f.codice = a.fattura
        inner join prodotto p on a.prodotto = p.codice
        inner join negozio n on f.negozio = n.codice
        inner join cliente c on f.cliente = c.codice
    where f.cliente = given_cliente
    GROUP BY f.codice, n.nome, c.username;
end;
$$ LANGUAGE plpgsql;
```

Viste e Viste Materializzate

Viste per le Statistiche

Per la visualizzazione delle statistiche di negozi, fornitori e prodotti ho la base di dati implementa delle viste, ovvero negozio_stats, fornitore_stats e prodotto_stats.

Essendo molto simili tra loro vediamo l'esempio di negozio_stats:

```
CREATE OR REPLACE VIEW myshop.negozio_stats
AS WITH stats_vende AS (
   SELECT vende.negozio,
       count(*) AS num_prodotti,
       avg(vende.prezzo) AS media_prezzi
   FROM myshop.vende JOIN myshop.prodotto ON vende.prodotto = prodotto.codice
      AND prodotto.data_eliminazione IS NULL
     GROUP BY vende.negozio ),
stats_fattura AS (
    SELECT fattura.negozio,
    count(*) AS num_fatture,
    sum(fattura.spesa) AS guadagno_totale
  FROM myshop.fattura
  GROUP BY fattura.negozio
)
SELECT n.codice,
    n.nome,
    COALESCE(sv.num_prodotti, 0::bigint) AS numero_prodotti,
    COALESCE(sf.num_fatture, 0::bigint) AS numero_fatture,
    COALESCE(sf.guadagno_totale, 0::numeric) AS guadagno_totale,
    sv.media_prezzi
FROM myshop.negozio n
    LEFT JOIN stats_vende sv ON n.codice = sv.negozio
```

```
    LEFT JOIN stats_fattura sf ON n.codice = sf.negozio
WHERE n.data_eliminazione IS NULL;
```

Vista Clienti Tesserati

Questa vista associa ad ogni cliente non eliminato la propria tessera attiva:

```
CREATE OR REPLACE VIEW myshop.clienti_tessera
AS SELECT c.codice,
    c.username,
    c.email,
    c.cf,
    t.punti,
    t.negozio,
    t.data_rilascio
   FROM myshop.cliente c
     LEFT JOIN myshop.tessera t ON c.codice = t.cliente AND t.data_disattivazione IS NULL
  WHERE c.data_eliminazione IS NULL
  ORDER BY c.username;
```

Vista Materializzata 300 punti

Per mantenere un elenco aggiornato dei clienti che hanno una tessera con un saldo punti superiore a 300 punti ho implementato una vista materializzata, aggiornata manualmente tramite un trigger.

DDL:

```
CREATE TABLE myshop.mv_clienti_300_punti (
    cliente bpchar(8) NOT NULL,
    username varchar(50) NOT NULL,
    tessera bpchar(8) NOT NULL,
    punti int4 NOT NULL,
    PRIMARY KEY (cliente, tessera),
    FOREIGN KEY (cliente) REFERENCES myshop.cliente(codice),
    FOREIGN KEY (tessera) REFERENCES myshop.tessera(codice)
);
```

Questa tabella viene aggiornata dal seguente trigger ogni volta che viene effettuato un update su tessera. In caso durante l'update la tessera viene disattivata allora questa viene eliminata dalla vista materializzata.

```
CREATE OR REPLACE FUNCTION myshop.aggiorna_mv_punti() RETURNS trigger AS $$
declare
    nome_cliente myshop.cliente.username%TYPE;
BEGIN
    if old.data_disattivazione is null and new.data_disattivazione is not null then
```

```
        delete from myshop.mv_clienti_300_punti where tessera = new.codice;
        return new;
    end if;

    IF NEW.punti >= 300 THEN
        perform 1 from myshop.mv_clienti_300_punti where tessera = new.codice;

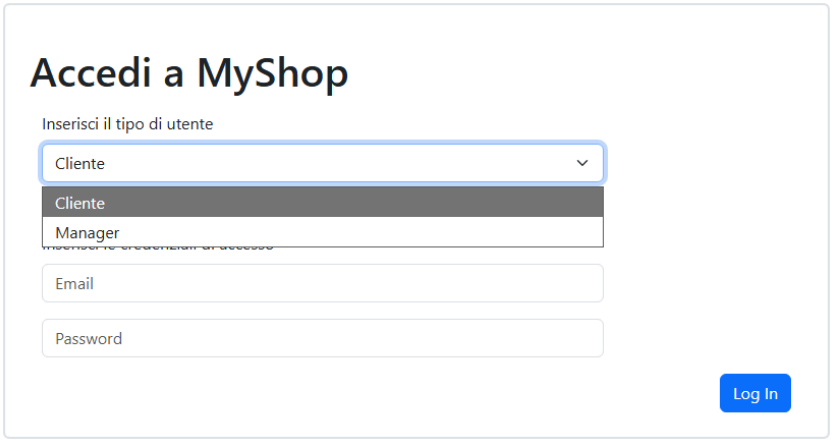
        if found then
            update myshop.mv_clienti_300_punti set punti = new.punti where tessera = new.codice;
        else
            select username into nome_cliente from myshop.cliente where codice = new.cliente;
            insert into myshop.mv_clienti_300_punti(cliente, username, tessera, punti) values (new.cliente,
        end if;
    ELSE
      DELETE FROM mv_clienti_300_punti WHERE tessera = NEW.codice;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Prove di Funzionamento

login.php

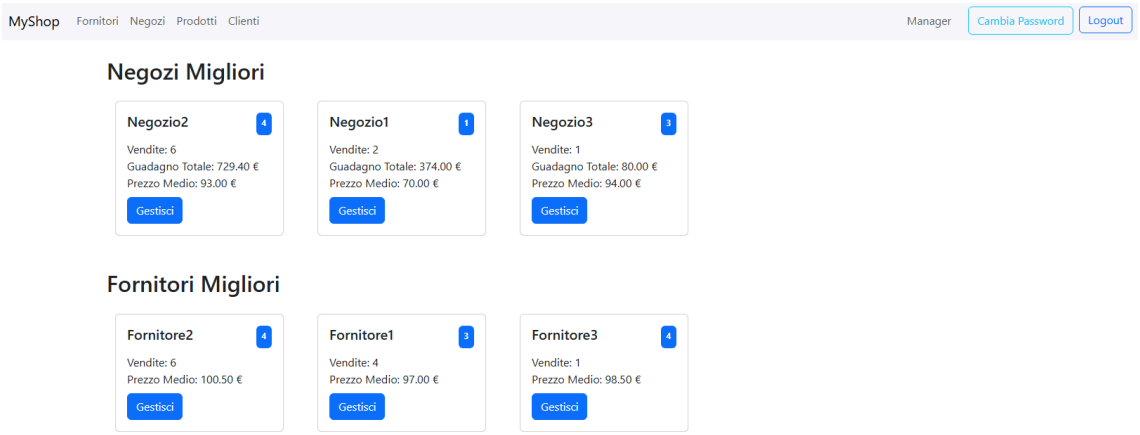
La pagina di log-in contiene un semplice form che oltre a richiedere email e password, richiede anche il ruolo con cui si vuole fare l'accesso, tramite un tag `select`.



homeManager.php

Questa é la pagina alla quale si viene portati dopo aver effettuato l'accesso come manager.

Nel corpo della pagina troviamo i 5 migliori negozi, decisi in base al guadagno totale e dal numero di prodotti in vendita, e i 5 migliori fornitori, decisi secondo il numero di ordini spediti ed il numero di prodotti che offrono.



Possiamo notare come nella navbar abbiamo la possibilità di visualizzare tutti i fornitori, negozi, prodotti o negozi. Oltre alla possibilità di cambiare la password ed effettuare il log-out.

Prendendo come esempio sempre i negozi, essendo simili tra loro i diversi tab, vediamo la seguente pagina.

homeManager.php?mod=shops

Il corpo della pagina cambia nel seguente modo, in questo tab siamo in grado di vedere tutti i negozi attivi nella base di dati. Schiacciando il bottone *Nuovo Negozio* si apre un modale con il form per l'inserimento di un nuovo negozio.

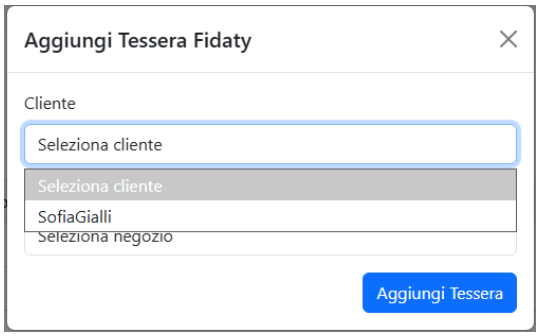


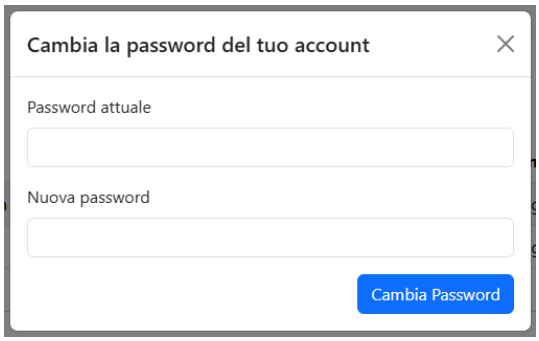


homeManager.php?mod=customers

La pagina dei clienti é invece impostata in generale per la gestione di tutti gli utenti, infatti ogni azione di modifica, eliminazione e aggiunta di tessera richiede di specificare tramite ad un tag `select` su quale cliente vogliamo agire.
Possiamo vedere i dati di ciascun utente e le tessere a loro assegnate, in caso assegnarne anche una.
Nella seconda sezione possiamo vedere gli utenti che hanno più di 300 punti sulla tessera.





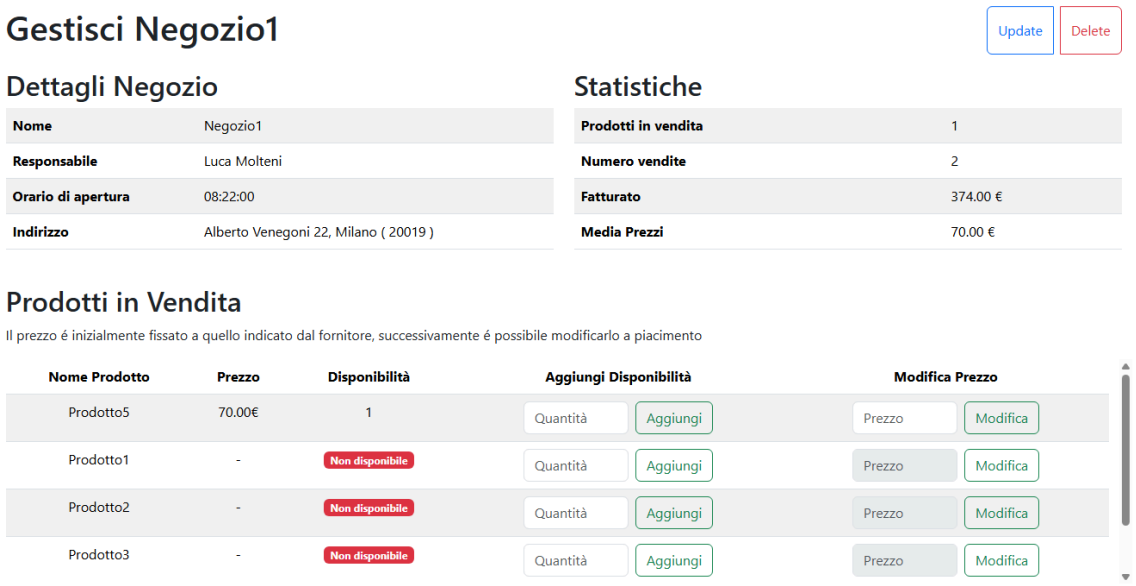


manageShop.php?id=NE000001

Questa pagina permette la gestione di un determinato negozio, ottenuto dal parametro `id` nell'url. Presenta inizialmente le informazioni e statistiche sul negozio, con la possibilità di modificare le informazioni del negozio oppure eliminarlo tramite i due pulsanti che attivano dei modali.

Nella sezione *Prodotti in Vendita* troviamo una lista con tutti i prodotti nella base di dati (non eliminati), troviamo prima quelli che il negozio vende e successivamente gli altri.
Il manager é in grado tramite la colonna
Aggiungi Disponibilità della tabella di rifornire il negozio, se il prodotto non era già in vendita nel negozio il suo prezzo é inizialmente impostato a quello indicato dal fornitore. Per modificare il prezzo possiamo usare invece il form nella colonna *Modifica Prezzo*, attivo solo sui prodotti che il negozio vende.

NOTA: Nel caso della pagina Gestisci Fornitore il prezzo é inizialmente impostato a 0, successivamente si può modificare a piacimento.



Al di sotto troviamo altre due sezioni, la prima é *Storico Fatture* che permette di visualizzare tutte le fatture che sono state emesse dal negozio. La seconda sezione é *Tessere Rilasciate* che invece permette di visualizzare tutte le tessere attive e disattivate che sono state rilasciate dal negozio.



Vediamo una prova del funzionamento della funzione ordina_prodotto() che si occupa di trovare il fornitore che vende il prodotto al miglior prezzo.

Tra le parentesi é indicata la disponibilità in base alla quantità che il negozio acquista.
Se un entrata é verde allora il negozio dovrebbe acquistare il prodotto p* dal fornitore f*.

	F1	F2	F3	Acquisto
P1	100 (giusta)	120 (sopra)	80 (sotto)	3
P2	90 (sotto)	90 (sopra)	90 (giusta)	2
P3	111 (sotto)	122 (sopra)	144 (sopra)	4
P4	80 (sopra)	—	—	1
P5	90 (sotto)	70 (sopra)	80 (giusta)	5

Dopo aver eseguito l'ordine di ognuno dei prodotti, la tabella ordine diventa correttamente:

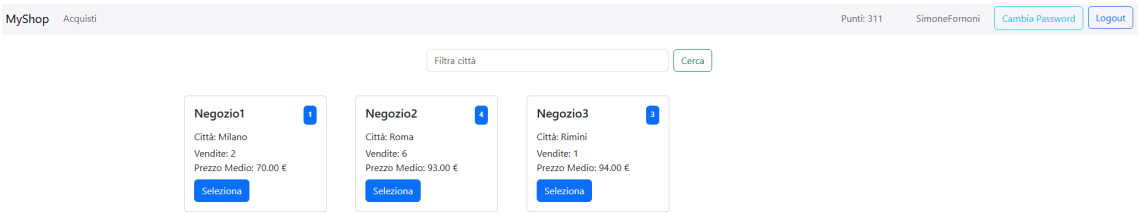
	codice	data_consegna	quantita	negozio	fornitore	prodotto
1	OR000001	2025-07-08 16:20:33.840	3	NE000002	FO000001	PR000001
2	OR000002	2025-07-08 16:21:03.627	2	NE000002	FO000002	PR000002
3	OR000003	2025-07-08 16:21:32.224	4	NE000002	FO000002	PR000003
4	OR000004	2025-07-08 16:21:48.220	1	NE000002	FO000001	PR000004
5	OR000005	2025-07-08 16:22:07.301	5	NE000002	FO000002	PR000005

homeCustomer.php

Questa é la pagina alla quale si viene portati dopo aver effettuato l'accesso come cliente.

Sulla navbar oltre ai tab e pulsanti troviamo anche i punti che ha attualmente sulla tessera il cliente.

Nel corpo della pagina troviamo tutti i negozi attivi nella base di dati e possiamo filtrarli in base alla città. Come si vede nella seconda immagine se filtriamo per città che iniziano con "R", troviamo



homeCustomer.php?action=search

É possibile filtrare i negozi secondo la città, ad esempio se cerchiamo i negozi che iniziano per "R", troviamo:



homeCustomer.php?mod=purchases

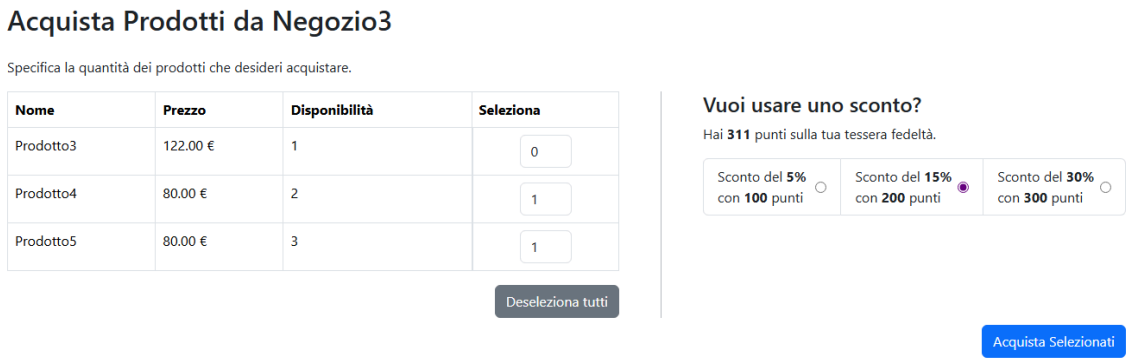
Spostandosi nel tab *Acquisti* il cliente può visualizzare tutti gli acquisti fatti.



buy.php?id=NE000003

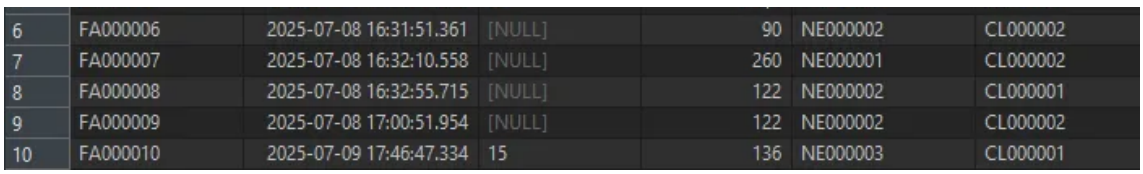
Questa pagina permette al cliente di acquistare i prodotti in vendita di un negozio.

Il cliente può specificare la quantità che vuole acquistare per ciascun prodotto, verranno presi in considerazione solamente quelli con quantità diversa da 0. Inoltre il cliente può scegliere di utilizzare uno tra gli sconti possibili in base ai punti nella sua tessera.



Vediamo cosa succede se inseriamo questa fattura (FA000010) :

Tabella fattura:



6	FA000006	2025-07-08 16:31:51.361	[NULL]	90	NE000002	CL000002
7	FA000007	2025-07-08 16:32:10.558	[NULL]	260	NE000001	CL000002
8	FA000008	2025-07-08 16:32:55.715	[NULL]	122	NE000002	CL000001
9	FA000009	2025-07-08 17:00:51.954	[NULL]	122	NE000002	CL000002
10	FA000010	2025-07-09 17:46:47.334	15	136	NE000003	CL000001

Tabella acquisto:

10	FA000007	PR000005	70	2
11	FA000008	PR000003	122	1
12	FA000009	PR000003	122	1
13	FA000010	PR000004	80	1
14	FA000010	PR000005	80	1

Vista clienti_tessera:

	codice	username	email	cf	punti	negozio	data_rilascio
1	CL000001	SimoneFornoni	simonefornoni@gmail.com	FRNSMN04T02E514U	247	NE000001	2025-07-08 00:00:00.000
2	CL000002	SofiaGialli	sofiagialli@gmail.com	SOFGLL22T08E782N	0	NE000003	2025-07-09 16:47:47.907

Vista Materializzata mv_clienti_300_punti:

	cliente	username	tessera	punti